

Software Quality Assurance – Software Engineering

Software Quality Assurance (SQA) is simply a way to assure quality in the software. It is the set of activities that ensure processes, procedures as well as standards are suitable for the project and implemented correctly.

Software Quality Assurance is a process that works parallel to Software Development. It focuses on improving the process of development of software so that problems can be prevented before they become major issues. Software Quality Assurance is a kind of Umbrella activity that is applied throughout the software process.

Elements of Software Quality Assurance (SQA)

1. **Standards:** The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents. The job of SQA is to ensure that standards that have been adopted are followed and that all work products conform to them.
2. **Reviews and audits:** Technical reviews are a quality control activity performed by software engineers for software engineers. Their intent is to uncover errors. Audits are a type of review performed by SQA personnel (people employed in an organization) with the intent of ensuring that quality guidelines are being followed for software engineering work.
3. **Testing:** [Software testing](#) is a quality control function that has one primary goal—to find errors. The job of SQA is to ensure that testing is properly planned and efficiently conducted for primary goal of software.
4. **Error/defect collection and analysis :** SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.
5. **Change management:** SQA ensures that adequate change management practices have been instituted.
6. **Education:** Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders. The SQA organization takes the lead in software process improvement which is key proponent and sponsor of educational programs.
7. **Security management:** SQA ensures that appropriate process and technology are used to achieve software security.
8. **Safety:** SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.
9. **[Risk management](#) :** The SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

Software Quality Assurance (SQA) focuses

The Software Quality Assurance (SQA) focuses on the following

- **Software's portability:** Software's **portability** refers to its ability to be easily transferred or adapted to different environments or platforms without needing significant modifications. This ensures that the software can run efficiently across various systems, enhancing its accessibility and flexibility.
- **software's usability:** **Usability** of software refers to how easy and intuitive it is for users to interact with and navigate through the application. A high level of usability ensures that users can effectively accomplish their tasks with minimal confusion or frustration, leading to a positive user experience.
- **software's reusability:** **Reusability** in software development involves designing components or modules that can be reused in multiple parts of the software or in different projects. This promotes efficiency and reduces development time by eliminating the need to reinvent the wheel for similar functionalities, enhancing productivity and maintainability.
- **software's correctness:** **Correctness** of software refers to its ability to produce the desired results under specific conditions or inputs. Correct software behaves as expected without errors or unexpected behaviors, meeting the requirements and specifications defined for its functionality.
- **software's maintainability:** **Maintainability** of software refers to how easily it can be modified, updated, or extended over time. Well-maintained software is structured and documented in a way that allows developers to make changes efficiently without introducing errors or compromising its stability.
- **software's error control:** **Error control** in software involves implementing mechanisms to detect, handle, and recover from errors or unexpected situations gracefully. Effective error control ensures that the software remains robust and reliable, minimizing disruptions to users and providing a smoother experience overall.

Software Quality Assurance (SQA) Include

1. A quality management approach.
2. Formal technical reviews.
3. Multi testing strategy.
4. Effective software engineering technology.
5. Measurement and reporting mechanism.

Major Software Quality Assurance (SQA) Activities

1. **SQA Management Plan:** Make a plan for how you will carry out the SQA throughout the project. Think about which set of software engineering activities are the best for project. check level of SQA team skills.
2. **Set The Check Points:** SQA team should set checkpoints. Evaluate the performance of the project on the basis of collected data on different check points.
3. **Measure Change Impact:** The changes for making the correction of an error sometimes re introduces more errors keep the measure of impact of change on project. Reset the new change to check the compatibility of this fix with whole project.
4. **Multi testing Strategy:** Do not depend on a single testing approach. When you have a lot of testing approaches available use them.

5. **Manage Good Relations:** In the working environment managing good relations with other teams involved in the project development is mandatory. Bad relation of SQA team with programmers team will impact directly and badly on project. Don't play politics.
6. **Maintaining records and reports:** Comprehensively document and share all QA records, including test cases, defects, changes, and cycles, for stakeholder awareness and future reference.
7. **Reviews software engineering activities:** The SQA group identifies and documents the processes. The group also verifies the correctness of software product.
8. **Formalize deviation handling:** Track and document software deviations meticulously. Follow established procedures for handling variances.

Benefits of Software Quality Assurance (SQA)

1. SQA produces high quality software.
2. High quality application saves time and cost.
3. SQA is beneficial for better reliability.
4. SQA is beneficial in the condition of no maintenance for a long time.
5. High quality commercial software increase market share of company.
6. Improving the process of creating software.
7. Improves the quality of the software.
8. It cuts maintenance costs. Get the release right the first time, and your company can forget about it and move on to the next big thing. Release a product with chronic issues, and your business bogs down in a costly, time-consuming, never-ending cycle of repairs.

Software Quality

Software quality is defined in different ways but here it means the conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software.

Following are the quality management system models under which the software system is created is normally based:

1. CMMI
2. Six Sigma
3. ISO 9000

Note: There may be many other models for quality management, but the ones mentioned above are the most popular.

Software Quality Assurance (SQA) Activities

Software Quality Assurance is composed of a variety of tasks associated with two different fields:

1. The software engineers who do technical work.
2. SQA group that has responsibility for quality assurance planning, oversight, record keeping, analysis, and reporting.

Basically, software engineers address quality (and perform quality assurance and quality control activities) by applying solid technical methods and measures, conducting formal technical reviews, and performing well-planned software testing.

Prepares an SQA Plan for a Project

This type of plan is developed during project planning and is reviewed by all interested parties. The quality assurance activities performed by the software engineering team and the SQA group are governed by the plan. The plan identifies:

- Evaluations to be performed.
- Audits and reviews to be performed.
- Standards that are applicable to the project.
- Procedures for error reporting and tracking.
- All the documents to be produced by the SQA group.
- The total amount of feedback provided to the software project team.

Every Participant in the Development of the Project's Software Process Description

The software team has to select a process for the work to be performed. The process description is reviewed by the SQA group for conformance with organizational policy, internal software standards, externally imposed standards, and other parts of the software project plan.

Software Engineering Activities are Reviewed to Verify Compliance with the Defined Software Process

The work of the SQA group is to identify documents and to track deviations from the process and verify that corrections have been made.

Financial Inspection Designated Software Work Products to Verify Conformance with those Defined as Part of the Software Process

1. The SQA group reviews selected work products; identifies, documents, and tracks deviations.
2. Verifies that corrections have been made.
3. Periodically reports the results of its work to the project manager.

Ensures that Deviations in Products are Documented and Handled According to a Documented Procedure

Deviations in software work and work products may be faced in the project plan, process description, applicable standards, or technical work products.

Records Reports to Senior Management and Non-compliance

Noncompliance items are tracked until they are resolved.

DEFINITIONS OF SOFTWARE RELIABILITY

Software reliability is defined as the probability of failure-free operation of a software system for a specified time in a specified environment. The key elements of the definition include probability of failure-free operation, length of time of failure-free operation and the given execution environment. Failure intensity is a measure of the reliability of a software system operating in a given environment. Example: An air traffic control system fails once in two years.

Factors Influencing Software Reliability

- A user's perception of the reliability of a software depends upon two categories of information.
 - The number of faults present in the software.

- The way users operate the system. This is known as the *operational profile*.
- The fault count in a system is influenced by the following.
 - Size and complexity of code.
 - Characteristics of the development process used.
 - Education, experience, and training of development personnel.
 - Operational environment.

Applications of Software Reliability

The applications of software reliability includes

- **Comparison of software engineering technologies.**
 - What is the cost of adopting a technology?
 - What is the return from the technology — in terms of cost and quality?
- **Measuring the progress of system testing** –The failure intensity measure tells us about the present quality of the system: high intensity means more tests are to be performed.
- **Controlling the system in operation** –The amount of change to a software for maintenance affects its reliability.
- **Better insight into software development processes** – Quantification of quality gives us a better insight into the development processes.

FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

System functional requirements may specify error checking, recovery features, and system failure protection. System reliability and availability are specified as part of the non-functional requirements for the system.

SYSTEM RELIABILITY SPECIFICATION

- Hardware reliability focuses on the probability a hardware component fails.
- Software reliability focuses on the probability a software component will produce an incorrect output.
- The software does not wear out and it can continue to operate after a bad result.
- Operator reliability focuses on the probability when a system user makes an error.

FAILURE PROBABILITIES

If there are two independent components in a system and the operation of the system depends on them both then, $P(S) = P(A) + P(B)$

If the components are replicated then the probability of failure is $P(S) = P(A)^n$ which means that all components fail at once.

FUNCTIONAL RELIABILITY REQUIREMENTS

- The system will check all operator inputs to see that they fall within their required ranges.
- The system will check all disks for bad blocks each time it is booted.
- The system must be implemented in using a standard implementation of Ada.

NON-FUNCTIONAL RELIABILITY SPECIFICATION

The required level of reliability must be expressed quantitatively. Reliability is a dynamic system attribute. Source code reliability specifications are meaningless (e.g. N faults/1000 LOC). An appropriate metric should be chosen to specify the overall system reliability.

HARDWARE RELIABILITY METRICS

Hardware metrics are not suitable for software since its metrics are based on notion of component failure. Software failures are often design failures. Often the system is available after the failure has occurred. Hardware components can wear out.

SOFTWARE RELIABILITY METRICS

Reliability metrics are units of measure for system reliability. System reliability is measured by counting the number of operational failures and relating these to demands made on the system at the time of failure. A long-term measurement program is required to assess the reliability of critical systems.

PROBABILITY OF FAILURE ON DEMAND

The probability system will fail when a service request is made. It is useful when requests are made on an intermittent or infrequent basis. It is appropriate for protection systems where service requests may be rare and consequences can be serious if service is not delivered. It is relevant for many safety-critical systems with exception handlers.

RATE OF FAULT OCCURRENCE

Rate of fault occurrence reflects upon the rate of failure in the system. It is useful when system has to process a large number of similar requests that are relatively frequent. It is relevant for operating systems and transaction processing systems.

RELIABILITY METRICS

- **Probability of Failure on Demand (PoFoD)**
 - $PoFoD = 0.001$.
 - For one in every 1000 requests the service fails per time unit.
- **Rate of Fault Occurrence (RoCoF)**
 - $RoCoF = 0.02$.
 - Two failures for each 100 operational time units of operation.
- **Mean Time to Failure (MTTF)**
 - The average time between observed failures (aka MTBF)
 - It measures time between observable system failures.
 - For stable systems $MTTF = 1/RoCoF$.
 - It is relevant for systems when individual transactions take lots of processing time (e.g. CAD or WP systems).
- **Availability = $MTBF / (MTBF + MTTR)$**
 - MTBF = Mean Time Between Failure
 - MTTR = Mean Time to Repair
- **Reliability = $MTBF / (1 + MTBF)$**

TIME UNITS

Time units include:

- **Raw Execution Time** which is employed in non-stop system
- **Calendar Time** is employed when the system has regular usage patterns
- **Number of Transactions** is employed for demand type transaction systems

AVAILABILITY

Availability measures the fraction of time system is really available for use. It takes repair and restart times into account. It is relevant for non-stop continuously running systems (e.g. traffic signal).

FAILURE CLASSIFICATION

Failure can be classified as the following

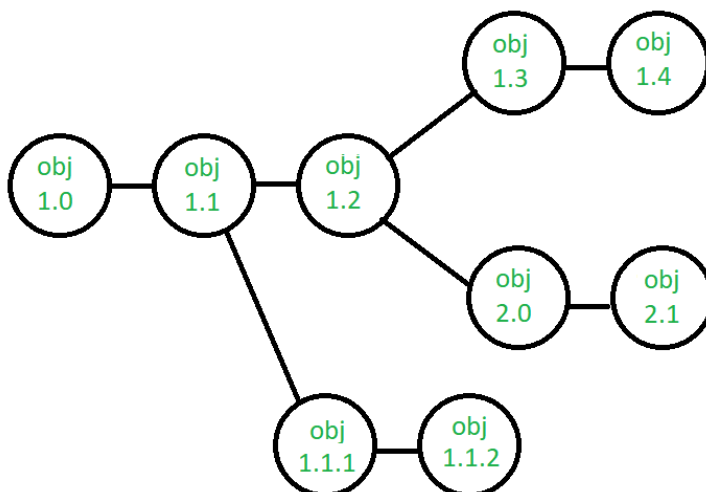
- **Transient** – only occurs with certain inputs.
- **Permanent** – occurs on all
- **Recoverable** – system can recover without operator help.
- **Unrecoverable** – operator has to help.
- **Non-corrupting** – failure does not corrupt system state or d
- **Corrupting** – system state or data are altered.

System configuration management – Software Engineering

Whenever software is built, there is always scope for improvement and those improvements bring picture changes. Changes may be required to modify or update any existing solution or to create a new solution for a problem. Requirements keep on changing daily so we need to keep on upgrading our systems based on the current requirements and needs to meet desired outputs. Changes should be analyzed before they are made to the existing system, recorded before they are implemented, reported to have details of before and after, and controlled in a manner that will improve quality and reduce error. This is where the need for System Configuration Management comes. **System Configuration Management (SCM)** is an arrangement of exercises that controls change by recognizing the items for change, setting up connections between those things, making/characterizing instruments for overseeing diverse variants, controlling the changes being executed in the current framework, inspecting and revealing/reporting on the changes made. It is essential to control the changes because if the changes are not checked legitimately then they may wind up undermining a well-run programming. In this way, SCM is a fundamental piece of all project management activities.

Processes involved in SCM – Configuration management provides a disciplined environment for smooth control of work products. It involves the following activities:

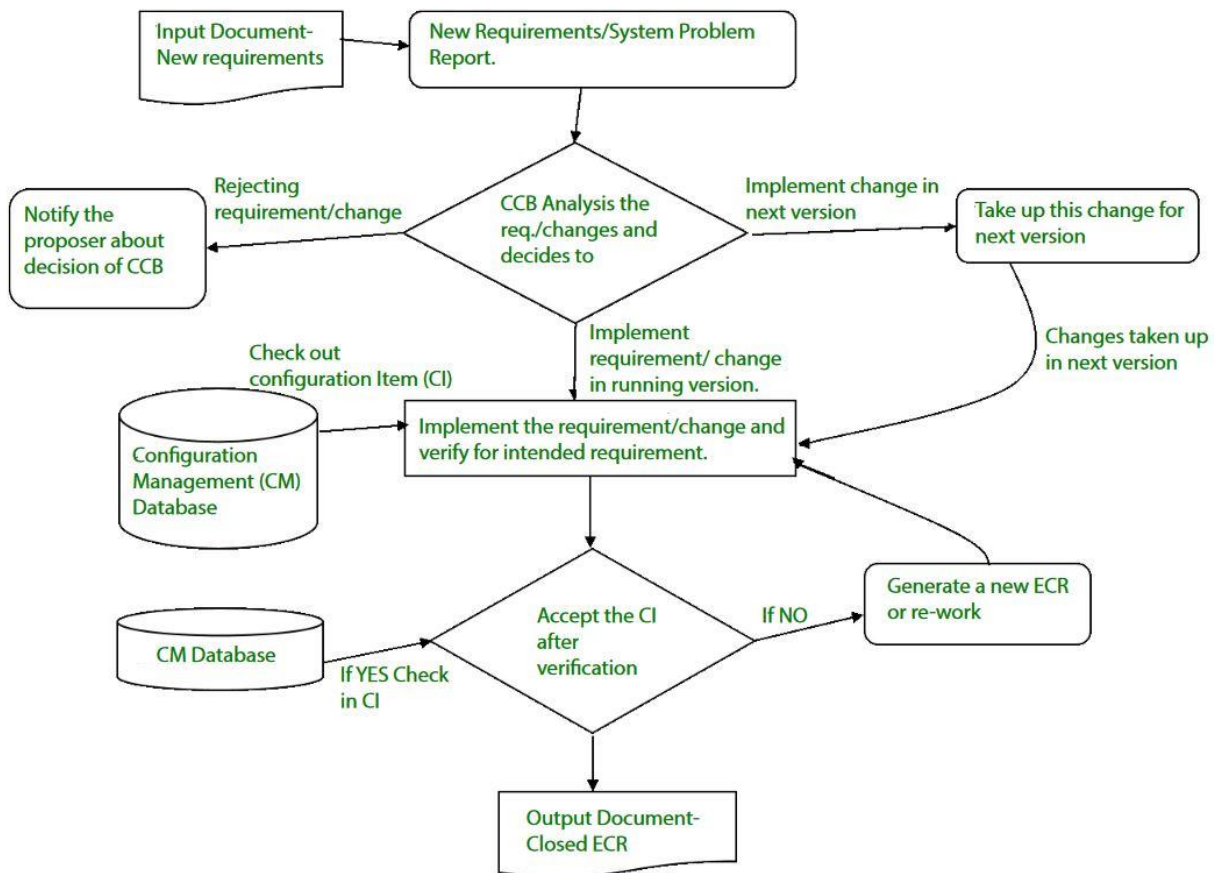
1. **Identification and Establishment** – Identifying the configuration items from products that compose baselines at given points in time (a baseline is a set of mutually consistent Configuration Items, which has been formally reviewed and agreed upon, and serves as the basis of further development). Establishing relationships among items, creating a mechanism to manage multiple levels of control and procedure for the change management system.
2. **Version control** – Creating versions/specifications of the existing product to build new products with the help of the SCM system. A description of the version is given below:



2. Suppose after some changes, the version of the configuration object changes from 1.0 to 1.1. Minor corrections and changes result in versions 1.1.1 and 1.1.2, which is followed by

a major update that is object 1.2. The development of object 1.0 continues through 1.3 and 1.4, but finally, a noteworthy change to the object results in a new evolutionary path, version 2.0. Both versions are currently supported.

3. **Change control** – Controlling changes to Configuration items (CI). The change control process is explained in Figure below:



3. A change request (CR) is submitted and evaluated to assess technical merit, potential side effects, the overall impact on other configuration objects and system functions, and the projected cost of the change. The results of the evaluation are presented as a change report, which is used by a change control board (CCB) —a person or group who makes a final decision on the status and priority of the change. An engineering change Request (ECR) is generated for each approved change. Also, CCB notifies the developer in case the change is rejected with proper reason. The ECR describes the change to be made, the constraints that must be respected, and the criteria for review and audit. The object to be changed is “checked out” of the project database, the change is made, and then the object is tested again. The object is then “checked in” to the database and appropriate version control mechanisms are used to create the next version of the software.
4. **Configuration auditing** – A software configuration audit complements the formal technical review of the process and product. It focuses on the technical correctness of the

configuration object that has been modified. The audit confirms the completeness, correctness, and consistency of items in the SCM system and tracks action items from the audit to closure.

5. **Reporting** – Providing accurate status and current configuration data to developers, testers, end users, customers, and stakeholders through admin guides, user guides, FAQs, Release notes, Memos, Installation Guide, Configuration guides, etc.

System Configuration Management (SCM) is a software engineering practice that focuses on managing the configuration of software systems and ensuring that software components are properly controlled, tracked, and stored. It is a critical aspect of [software development](#), as it helps to ensure that changes made to a software system are properly coordinated and that the system is always in a known and stable state.

SCM involves a set of processes and tools that help to manage the different components of a software system, including source code, documentation, and other assets. It enables teams to track changes made to the software system, identify when and why changes were made, and manage the integration of these changes into the final product.